

TP-2-IM01

Mohamed Boudouh

October 2025

Transformation géométrique

Ce que je constate globalement est que l'interpolation par la méthode bilinéaire donne un résultat plus "lumineux" que l'interpolation par la méthode du plus proche voisin.

Je pense que ceci peut s'expliquer par le fait que les "nouvelles" valeurs calculées par l'interpolation par la méthode bilinéaire sont une moyenne pondérée des valeurs des voisins du pixel inconnu, alors que la méthode du plus proche voisin n'attribue qu'une valeur d'un voisin. Enfin, vu que je pense que l'histogramme de l'image initiale est plus concentré autour des valeurs basses que celles hautes, je pense que le résultat global du calcul par moyennes pondérées est plus "lumineux" que le résultat du calcul par la méthode des plus proche voisin.

Un point commun entre les deux méthodes est le floutage par rapport à l'image initiale vu que les deux méthodes d'interpolation peuvent être considérées comme des filtres passe-bas.

Pour les rotations successives de 45° , les bords sont effacés (je pense car ils étaient effacés dans les images intermédiaires). La qualité des images est aussi successivement détériorée.

Pour le zoom 0.5, je pense qu'un zoom peut être modélisé par un échantillonnage (suivi d'une translation) (Ceci n'est un cas particulier d'interpolation de plus proches voisins (en cas de voisins équidistants, l'algorithme peut choisir de sélectionner le même pixel 1 ou 2 ou 3 ou 4 fois, mais je pense que ces cas sont un peu similaires). D'où l'effet d'aliasing. Cet effet est moins remarquable dans le cas d'interpolation par la méthode bilinéaire (que je pense peut être modélisée par un filtrage passe-bas suivi par un échantillonnage et enfin une translation)

Dans le premier cas, on je pense qu'il peut être mieux de filtrer par un filtre passe-bas avant d'échantillonner.

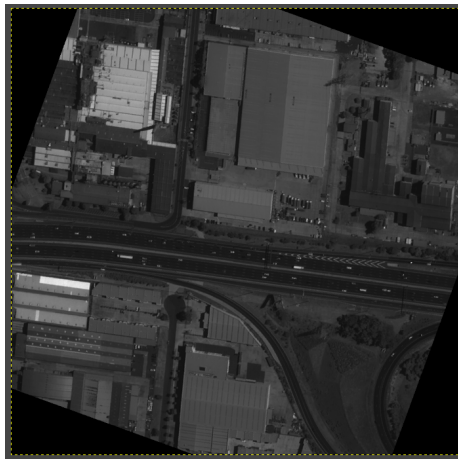


Figure 1: Image initiale

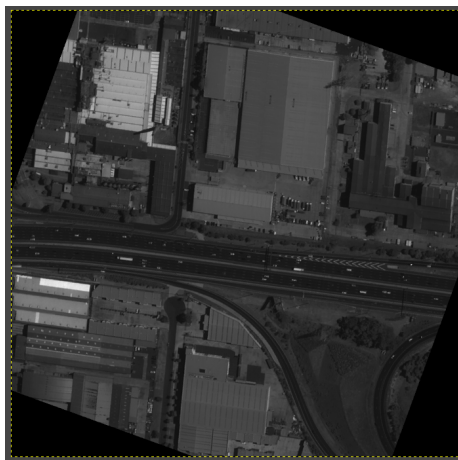


Figure 2: Image après rotation et interpolation pa la méthode du plus proche voisin

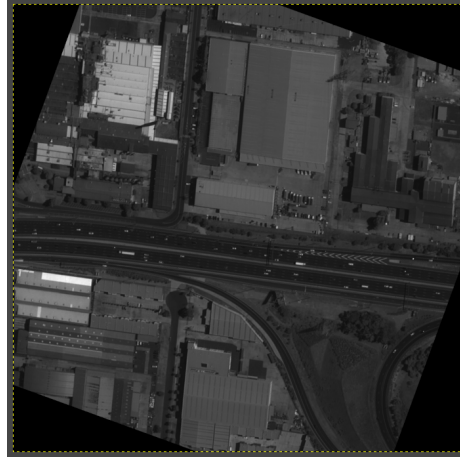


Figure 3: Image après rotation par interpolation par la méthode d'interpolation linéaire

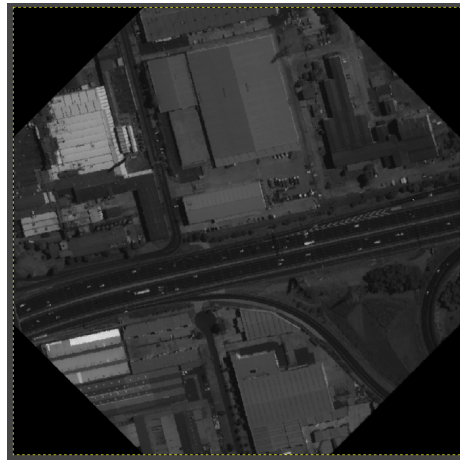


Figure 4: Image après 8 rotations de 45° (interpolation par la méthode du plus proche voisin)

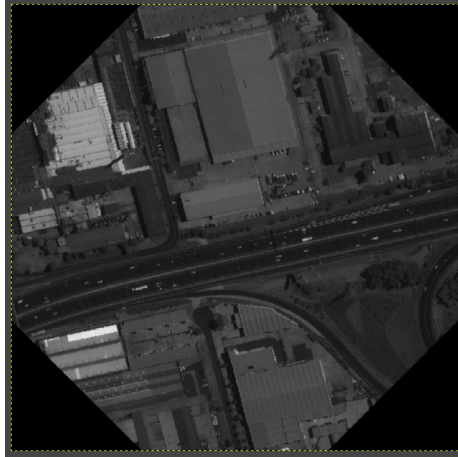


Figure 5: (Image après 8 rotations de 45° (interpolation par la méthode bilinéaire)

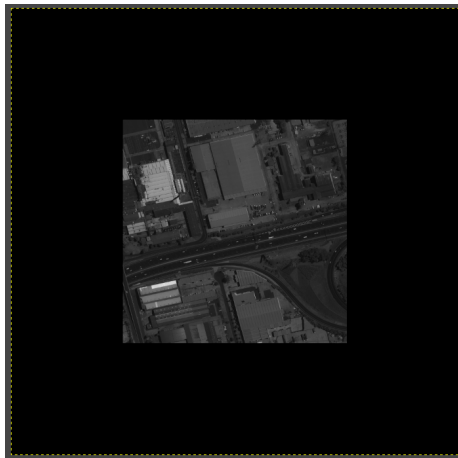


Figure 6: Zoom 0.5(Intéropolation par la méthode du plus proche voisin)

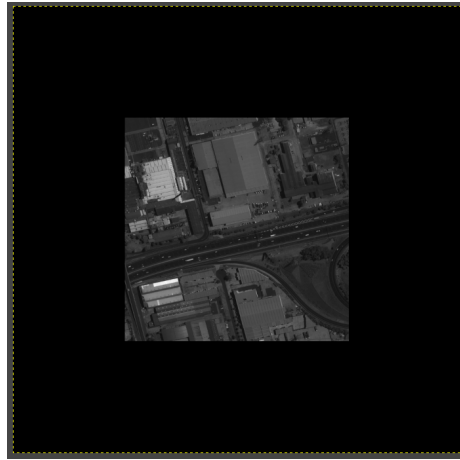


Figure 7: Zoom 0.5(Intérpolation par la méthode bilinéaire)



Figure 8: Aliasing plus clair(Intérpolation par la méthode du plus proche voisin)



Figure 9: Aliasing plus clair (qui me paraît moins sévère que celui juste au dessus) (Interpolation par la méthode bilinéaire)

Filtrage linéaire et médian

-

La fonction renvoie un noyau de taille $2.5 \times s \times 2.5 \times s$, où s est l'écart type de la fonction gaussienne (dans les deux directions)

-

Le bruit étant uniforme sur l'image, on peut considérer que sa variance sur une partie de pixel de niveaux de gris constant (par exemple la partie centrale blanche) est une bonne estimation de la variance globale du bruit. Ceci étant pour éviter d'introduire la variance de l'image résultant des effets de bord dans notre estimation de la variance résultante du bruit

-

Le résultat du filtrage médian est plus bruité car le filtrage médian "a plus de chance" de garder plus de hautes fréquences.

-

Pour l'image 'pyra-impulse.tif', L'image résultante du filtrage médian me paraît plus fidèle à l'image originale que celle résultante du filtrage linéaire. En effet, Vu que les pixels de l'image ont des valeurs 'constantes' sur des régions de l'image, dès qu'on dépasse légèrement les bords, la valeur médiane de l'entourage du pixel devient égale à sa valeur originale. Le filtrage linéaire fait

éventuellement obtenir de nouvelles valeurs pour les pixels qui n'étaient pas dans l'image d'origine. // Pour le point lumineux en haut à droite du carré. Le filtrage linéaire donne une image dans laquelle il y a un point lumineux en haut à droite mais de luminosité remarquablement inférieure (car c'est une valeur moyenne). Le filtrage médian donne une image dans laquelle je ne vois pas de point lumineux en haut à droite du carré central (Je suppose donc que l'algorithme considère que la valeur médiane d'un ensemble à 2 éléments est sa valeur minimale)

Réstauration

-

Je pense que l'utilisation du filtre constant induit une perte définitive de l'information (filtre non inversible). Effectivement, en essayant d'appliquer le filtre inverse à une image filtrée linéairement par un masque constant, j'ai obtenu une image noire.

Par contre je pense que le filtre gaussien est théoriquement inversible mais je pense que son spectre s'annule numériquement à partir d'un certain rang. Ce qui peut expliquer l'apparition de nouvelles hautes fréquences dans l'image produite par la composition des deux filtrages (linéaire par masque gaussien et filtrage inverse adaptée à ce même masque).

En ajoutant du bruit, j'obtiens une image assez différente de l'image originale en effectuant le filtrage linéaire par masque gaussien puis son filtre inverse.

-

Si le filtre est inversible, on peut approximer l'image de départ par la transformée de Fourier inverse du rapport de la transformée de Fourier de l'image filtrée sur celle originale.

-

Le filtre donne généralement des résultats proches de l'image d'origine pour des valeurs faibles de λ . En particulier, je pense que le résultat est plus proche de l'image d'origine pour $\lambda = 0$ (comparé aux autres valeurs de $\lambda > 0$). Je pense que ceci peut s'expliquer par le fait qu'en augmentant la valeur de λ , on autorise des valeurs moins importantes pour les hautes fréquences, d'où des bords moins fins pour le carré blanc.

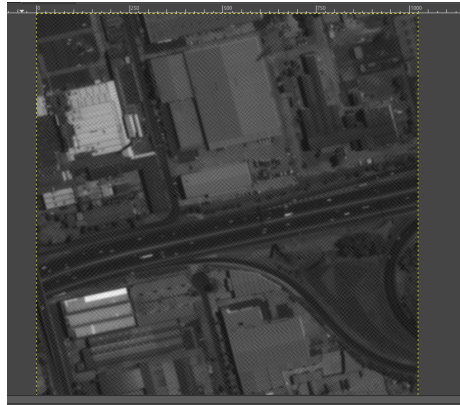


Figure 10: Image obtenue par filtrage inverse de l'image filtrée par un filtre gaussien

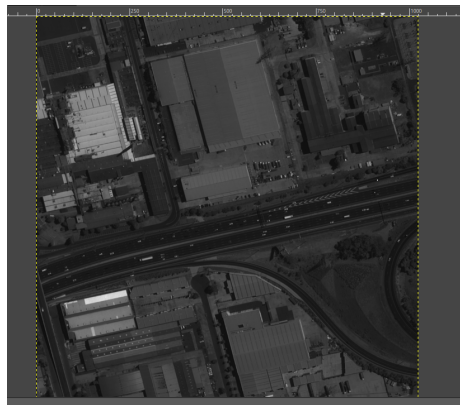


Figure 11: Image originale

Applications

Comparaison filtrage linéaire et médian

Honnêtement, je n'ai pas testé le code pour le moment.

je pense qu'on peut essayer itérativement des filtres constants de noyaux de taille qui augmente progressivement et choisir ensuite celle qui minimise une distance de l'image obtenue à celle obtenue par filtrage médian circulaire de taille 4. (Par exemple la somme (sur les pixels des carrés) des différences des valeurs pixel par pixel)

```
im3=noise(im3,5)
im_rétaburée_par_weiner= wiener(im3,K)
viewimage(im_rétaburée_par_weiner)
```

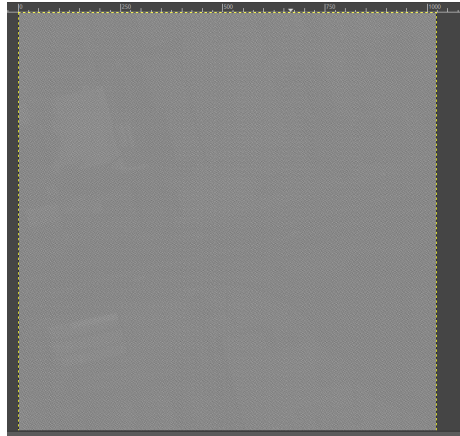


Figure 12: Résultat du filtrage inverse après bruitage de l'image filtré linéairement par un masque gaussien.

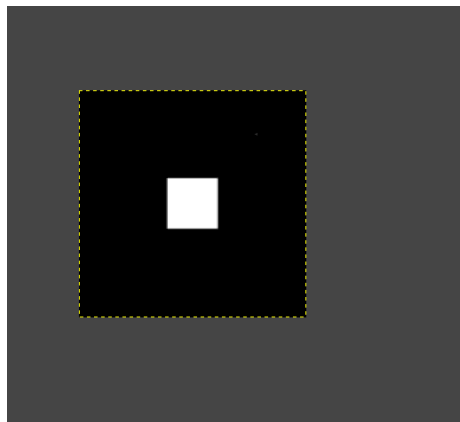


Figure 13: Résultat de restauration par le filtre de weiner pour $\lambda = 0$

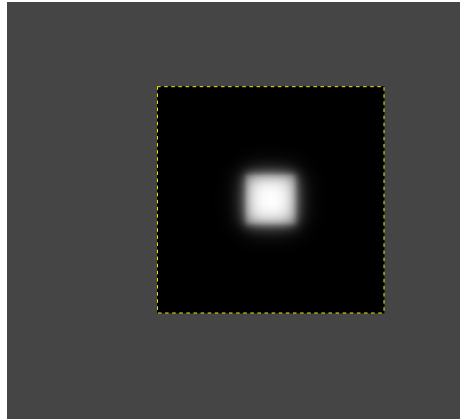


Figure 14: Résultat de restauration par le filtre de weiner pour $\lambda = 0.5$

```

im4= noise(im_2,5)
mask_1=get_cst_ker(4)
im4_denoised_by_median_filter=median_filter(im4,r=4)
def denoising_cst_kernel(im):
    L=[]
    minimal_difference = 10000
    for i in range(6):
        mask = get_cst_ker(i)
        im_denoised=filtre_lineaire(im,mask)
        difference= np.linalg.norm(im_denoised-im_denoised,2)
        if difference<minimal_difference:
            minimal_difference=difference
            L.append(im_denoised)

    return L[-1]

```

Calcul théorique du paramètre de restauration

Ci dessous le résultat de la tentative d'implémentation de ce filtre.

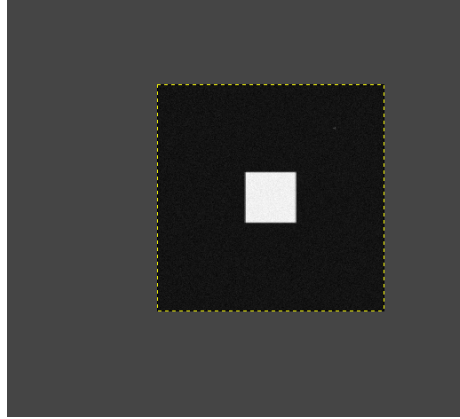


Figure 15: Implémentation du filtre de weiner où le paramètre λ est estimé par la fonction en recevant l'écart type du bruit comme paramètre(dans ce cas $\sigma_b = 5$).

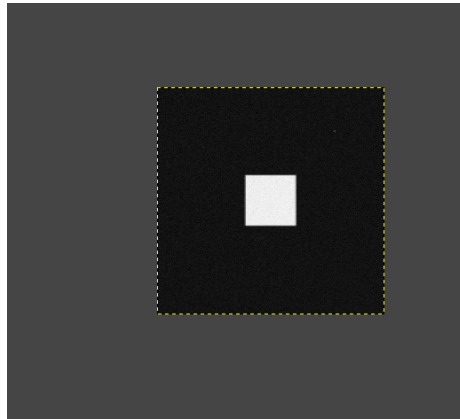


Figure 16: Implémentation du filtre de weiner où le paramètre λ est choisi égal à 0(dans ce cas $\sigma = 5$)